

Projeto ESTOQUE — Documentação técnica: Código

WINDEV 2025 · Update 1 · Gerado em 03/09/2026 · Idioma do projeto: pt-BR · Escopo: procedures globais, classes e eventos

Código do projeto

Inicialização do projeto

```
// Conexão com o HFSQL Client/Server; parâmetros vêm do arquivo estoque.ini
gsConexao is Connection
gsConexao.Provider = hAccessHFClientServer
gsConexao.Server = INIRead("HFSQL", "Servidor", "localhost:4900", fExeDir() +
"\estoque.ini")
gsConexao.Database = "ESTOQUE"
gsConexao.User = INIRead("HFSQL", "Usuario", "admin", fExeDir() + "\estoque.ini")
HChangeConnection("", gsConexao)
IF NOT HOpenConnection(gsConexao) THEN
    Error("Não foi possível conectar ao servidor HFSQL.", HErrorInfo())
EndProgram()
END
gnUsuarioLogado is int = 0
gsPerfil is string = ""
```

Conjunto de procedures globais: Regras

CalculaDesconto

```
// Regra comercial: desconto máximo por tipo de cliente.
// Cliente comum (C): até 15%. Cliente especial (E): até 25%.
// Vendedor não pode ultrapassar o teto; quem ultrapassa recebe erro e a venda não
fecha.
PROCEDURE CalculaDesconto(nSubtotal is currency, nPercentual is numeric, sTipoCliente is
string): currency
nTeto is numeric
SWITCH sTipoCliente
    CASE "E": nTeto = 25
    OTHER CASE: nTeto = 15
END
IF nPercentual > nTeto THEN
    Error("Desconto de " + nPercentual + "% ultrapassa o máximo de " + nTeto + "% para
este cliente.")
    RETURN -1
END
RETURN Round(nSubtotal * nPercentual / 100, 2)
```

ValidaEstoque

```
// Regra de estoque: não vende o que não tem no depósito escolhido.
// Retorna a quantidade disponível ou -1 quando o produto não tem saldo no depósito.
PROCEDURE ValidaEstoque(nIDProduto is int, nIDDeposito is int, nQuantidade is numeric):
boolean
HReadSeekFirst(SALDO, IDPRODUTO_IDDEPOSITO, [nIDProduto, nIDDeposito])
IF NOT HFound(SALDO) THEN
    Error("Produto sem saldo cadastrado neste depósito.")
    RETURN False
END
IF SALDO.QUANTIDADE < nQuantidade THEN
    Error("Estoque insuficiente. Disponível: " + NumToString(SALDO.QUANTIDADE, "12.3f"))
    RETURN False
```

```
END
RETURN True
```

BaixaEstoque

```
// Baixa transacional: ou baixa todos os itens da venda, ou nenhum.
PROCEDURE BaixaEstoque(nIDVenda is int): boolean
HTransactionStart(gsConexao)
HReadSeekFirst(ITEMVENDA, IDVENDA, nIDVenda)
WHILE HFound(ITEMVENDA)
    HReadSeekFirst(SALDO, IDPRODUTO_IDDEPOSITO, [ITEMVENDA.IDPRODUTO, VENDA.IDDEPOSITO])
    IF NOT HFound(SALDO) OR SALDO.QUANTIDADE < ITEMVENDA.QUANTIDADE THEN
        HTransactionCancel(gsConexao)
        Error("Baixa cancelada: item " + ITEMVENDA.IDPRODUTO + " sem saldo.")
        RETURN False
    END
    SALDO.QUANTIDADE -= ITEMVENDA.QUANTIDADE
    HModify(SALDO)
    HReadNext(ITEMVENDA, IDVENDA)
END
VENDA.STATUS = "F"
HModify(VENDA)
HTransactionEnd(gsConexao)
RETURN True
```

CalculaJurosAtraso

```
// Regra financeira: 2% ao mês pro rata die sobre o valor do título, a partir do dia
seguinte ao vencimento.
// Título pago no vencimento não tem juros.
PROCEDURE CalculaJurosAtraso(nValor is currency, dVencimento is Date, dPagamento is
Date): currency
nDias is int = DateDifference(dVencimento, dPagamento)
IF nDias <= 0 THEN RETURN 0
RETURN Round(nValor * 0.02 / 30 * nDias, 2)
```

ValidaCPF

```
// Validação de CPF pelos dois dígitos verificadores; rejeita sequências repetidas.
PROCEDURE ValidaCPF(sCPF is string): boolean
sCPF = NoSpace(Replace(Replace(sCPF, ".", ""), "-", ""))
IF Length(sCPF) <> 11 THEN RETURN False
IF sCPF = RepeatString(sCPF[[1]], 11) THEN RETURN False
nSoma, i are int
FOR i = 1 TO 9
    nSoma += Val(sCPF[[i]]) * (11 - i)
END
nDV1 is int = modulo(nSoma * 10, 11)
IF nDV1 = 10 THEN nDV1 = 0
IF nDV1 <> Val(sCPF[[10]]) THEN RETURN False
nSoma = 0
FOR i = 1 TO 10
    nSoma += Val(sCPF[[i]]) * (12 - i)
END
nDV2 is int = modulo(nSoma * 10, 11)
IF nDV2 = 10 THEN nDV2 = 0
RETURN nDV2 = Val(sCPF[[11]])
```

Janela WIN_Venda — eventos

Inicialização de WIN_Venda

```

VENDA.DATA = Today()
VENDA.VENDEDOR = gsUsuarioNome
VENDA.STATUS = "A"
TABLE_Itens.DeleteAll()
EDT_Subtotal = 0
EDT_Desconto = 0
EDT_Total = 0

```

Clique em BTN_AgregarItem

```

IF COMBO_Produto = 0 THEN
    Error("Escolha um produto.")
    ReturnToCapture(COMBO_Produto)
END
IF EDT_Quantidade <= 0 THEN
    Error("Quantidade deve ser maior que zero.")
    ReturnToCapture(EDT_Quantidade)
END
IF NOT ValidaEstoque(COMBO_Produto, COMBO_Deposito, EDT_Quantidade) THEN
    ReturnToCapture(EDT_Quantidade)
END
HReadSeekFirst(PRODUTO, IDPRODUTO, COMBO_Produto)
TableAddLine(TABLE_Itens, PRODUTO.CODIGO, PRODUTO.DESCRICAO, EDT_Quantidade,
PRODUTO.PRECO_VENDA, Round(EDT_Quantidade * PRODUTO.PRECO_VENDA, 2))
RecalculaTotais()

```

Procedure local RecalculaTotais

```

PROCEDURE RecalculaTotais()
nSub is currency = 0
FOR EACH ROW OF TABLE_Itens
    nSub += TABLE_Itens.COL_TotalItem
END
EDT_Subtotal = nSub
nDesc is currency = CalculaDesconto(nSub, EDT_PercDesconto, CLIENTE.TIPO)
IF nDesc < 0 THEN
    EDT_PercDesconto = 0
    nDesc = 0
END
EDT_Desconto = nDesc
EDT_Total = nSub - nDesc

```

Clique em BTN_Fechar

```

IF TABLE_Itens.Count = 0 THEN
    Error("A venda não tem itens.")
    RETURN
END
IF EDT_Total > CLIENTE.LIMITE_CREDITO AND CLIENTE.TIPO = "C" THEN
    IF NOT YesNo(No, "Total acima do limite de crédito do cliente. Fechar mesmo assim?")
    THEN RETURN
END
ScreenToFile(WIN_Venda)
HAdd(VENDA)
FOR EACH ROW OF TABLE_Itens
    ITEMVENDA.IDVENDA = VENDA.IDVENDA
    HReadSeekFirst(PRODUTO, CODIGO, TABLE_Itens.COL_Codigo)
    ITEMVENDA.IDPRODUTO = PRODUTO.IDPRODUTO
    ITEMVENDA.QUANTIDADE = TABLE_Itens.COL_Quantidade
    ITEMVENDA.PRECO_UNITARIO = TABLE_Itens.COL_Preco
    ITEMVENDA.TOTAL_ITEM = TABLE_Itens.COL_TotalItem
    HAdd(ITEMVENDA)
END

```

```

IF BaixaEstoque(VENDA.IDVENDA) THEN
    GeraTitulos(VENDA.IDVENDA, EDT_Total, COMBO_Parcelas)
    Info("Venda " + VENDA.IDVENDA + " fechada.")
    Close()
END

```

Procedure global GeraTitulos

```

// Parcelas iguais, vencendo a cada 30 dias; a diferença de arredondamento vai para a
última.
PROCEDURE GeraTitulos(nIDVenda is int, nTotal is currency, nParcelas is int)
nValorParcela is currency = Round(nTotal / nParcelas, 2)
nAcumulado is currency = 0
FOR i = 1 TO nParcelas
    TITULO.IDVENDA = nIDVenda
    TITULO.VENCIMENTO = DateSys() + 30 * i
    IF i = nParcelas THEN
        TITULO.VALOR = nTotal - nAcumulado
    ELSE
        TITULO.VALOR = nValorParcela
        nAcumulado += nValorParcela
    END
    HAdd(TITULO)
END

```

Janela WIN_Cliente — eventos

Saída de EDT_CPF

```

IF NOT ValidaCPF(EDT_CPF) THEN
    Error("CPF inválido.")
    ReturnToCapture(EDT_CPF)
END
HReadSeekFirst(CLIENTE, CPF, EDT_CPF)
IF HFound(CLIENTE) AND CLIENTE.IDCLIENTE <> EDT_IDCliente THEN
    Error("Já existe cliente com este CPF: " + CLIENTE.NOME)
    ReturnToCapture(EDT_CPF)
END

```

Clique em BTN_Salvar

```

ScreenToFile(WIN_Cliente)
IF CLIENTE.DATA_CADASTRO = "" THEN CLIENTE.DATA_CADASTRO = Today()
IF EDT_IDCliente = 0 THEN
    HAdd(CLIENTE)
ELSE
    HModify(CLIENTE)
END
Info("Cliente salvo.")
Close()

```